ALICIA VÁZQUEZ
Node.js:
Procesos e Hilos
Alicia Vázquez
@aliciaFPInf
BY NC SA

Punto de partida

Cuando haces una **API con Express,** Node gestiona las peticiones de forma **asíncrona**: mientras espera que llegue una respuesta de una base de datos o de otro servidor, sigue atendiendo otras peticiones. Eso funciona perfectamente para tareas que esperan cosas (leer ficheros, llamadas HTTP, consultas a BD).

El problema aparece cuando tienes una tarea que *no espera, sino que calcula*:

```
// Esto BLOQUEA Node completamente mientras calcula
// Nadie puede hacer peticiones a tu API durante este tiempo
app.get('/calcular', (req, res) => {
 let resultado = 0;
 for (let i = 0; i < 10_000_000_000; i++) {
   resultado += i;  //CPU a tope, Node congelado
 }
 res.json({ resultado });
});
```

En este caso, mientras ese bucle corre, la API no responde a nadie más. Eso es lo que queremos evitar.

Punto de partida → Restaurante

Imagina que nuestra API es un restaurante con un único cocinero (el hilo principal de Node):

- **Tarea asíncrona** (fetch, leer fichero): es como pedir algo al proveedor. El cocinero hace el pedido y mientras espera, cocina otras cosas. *Node lo gestiona solo.*
- **Tarea de CPU intensiva** (calcular, comprimir imagen, procesar CSV): es como pedirle al cocinero que pele 10.000 patatas él solo. Mientras pela, no cocina nada más. *Bloquea todo.*

La solución es contratar ayuda:

- **child_process** → contratar a otro cocinero completo, con su propia cocina. Más independiente, más pesado.
- **worker_threads** → contratar a un ayudante que trabaja en la misma cocina. Más ligero, comparte recursos.

01

Child_process

child_process: otro proceso independiente

¿Qué es?

Lanza un **programa separado**. Tiene su propia memoria, **su propio Node**. Es como abrir otra ventana del terminal y ejecutar otro script.

¿Cuándo usarlo?

- Al ejecutar un script externo (Python, bash, otro JS)
- Tareas completamente independientes que no necesitan compartir datos
- Cuando no importa el overhead de arrancar un proceso nuevo

Proceso: canales de comunicación de resultados

En un sistema operativo existen tres canales de comunicación estándar:

- **stdin (0)** → entrada → el proceso RECIBE datos
- **stdout (1)** → salida → el proceso ENVÍA datos/resultados
- **stderr (2)** → errores → el proceso ENVÍA logs/errores

stdout (Standard Output)

Es la salida normal del proceso. Cuando haces **console.log()** en Node, está escribiendo en **stdout**. Es lo que el proceso padre recibe en la variable stdout del callback de execFile. Si suamos console.log() en el proceso hijo esto va directo al **stdout** y al ser leído por el padre dará error!

```
console.log("tarea pesada: " + resultado); // → va a stdout
process.stdout.write(JSON.stringify({ resultado })); // → va a stdout.

// El padre recibía en stdout algo así:
// "tarea pesada: 15\n{"resultado":15}" // → JSON.parse() explota porque no es JSON válido
```

stderr (Standard Error)

Es el canal reservado para errores y logs de diagnóstico. Está separado de stdout precisamente para no mezclar datos con mensajes informativos. Lo que escribes aquí llega al **callback** de execFile en la variable **stderr**.

server.js

```
import { execFile } from 'child_process';

app.get('/sumar', (req, res) => {
 const numeros = [1, 2, 3, 4, 5];

 // Lanza el script en un proceso separado
 execFile('node', ['tarea-pesada.js', JSON.stringify(numeros)],
   (error, stdout, stderr) => {
     if (error) {
       return res.status(500).json({ error: 'Falló el proceso' });
     }
     const datos = JSON.parse(stdout);
     res.json(datos); // → { resultado: 15 }
   }
 );
 // Mientras tanto, Node sigue atendiendo otras peticiones
});
```

tarea-pesada.js

```
// Este fichero se ejecuta en un proceso separado
//console.log("tarea pesada" + resultado);
process.stderr.write("Tarea pesada arrancada\n");
const lista = process.argv[2] ?
JSON.parse(process.argv[2]) : [];
const resultado = lista.reduce((acc, n) => acc + n,
0);
process.stderr.write("resultado: " + resultado +
"\n");
process.stdout.write(JSON.stringify({ resultado }));
```

ANTES

Crear proyecto Node

```
npm init -y
```

Instalar Express

```
npm install express
```

PROCESO PADRE (server.js)

1. ARRANQUE: ENVÍO DE DATOS INICIALES (process.argv)
[Argumentos serializados con JSON.stringify()]

PROCESO HIJO (tarea-pesada.js)

2. FLUJO DE DATOS: ENVÍO DE RESULTADO (stdout)
[Salida Estándar - Éxito como JSON]

3. FLUJO DE DATOS: NOTIFICACIÓN DE ERROR (stderr)
[Salida de Error Estándar - Fallo]

API QUE DELEGA LA TAREA pesado.js
Llama execFile, NO Bloquea el Event Loop
Atiende otras peticiones simultáneamente

```
import { execFile } from 'child_process';

app.get('/sumar', (req, res) => {
  const numeros = [1, 2, 3, 4, 5];
  // Lanza el script en un proceso separado
  execFile('node', ['tarea-pesada.js', JSON.stringify(numeros)],
    (error, stdout, stderr) => {
    if (error) {
      return res.status(500).json({ error: 'Falló el proceso' });
    }
    const datos = JSON.parse(stdout);
    res.json(datos);                // → { resultado: 15 }
  );
  // ✅ Mientras tanto, Node sigue atendiendo otras peticiones
});
```

SCRIPT DE TRABAJO PESADO
Ejecuta el Cálculo Aislado, Retorna JSON
Simula Tarea Pesada

```
// Este fichero se ejecuta en un proceso separado
const lista = process.argv[2]  // recibe datos por argument
  ? JSON.parse(process.argv[2])
  : [];

// Simulamos tarea pesada: suma todos los números
const resultado = lista.reduce((acc, n) => acc + n, 0);

// Devuelve el resultado al proceso padre
process.stdout.write(JSON.stringify({ resultado }));
```

NODE.JS

ESQUEMA DE COMUNICACIÓN PADRE-HIJO

Ejercicio: La Lotería

Implementamos el backend de una lotería. Cuando alguien compra un boleto, el servidor lanza un proceso hijo que hace el "sorteo" (una operación "pesada") sin bloquear al resto de jugadores.

Crear dos ficheros:

servidor.js con Express que tenga dos rutas:

- GET /jugar/:nombre → lanza un proceso hijo para calcular si ese usuario gana
- GET /estado → responde inmediatamente con { servidor: "vivo" } para demostrar que Node no se ha bloqueado

sorteo.js (el proceso hijo) que recibe un nombre por argumento y:

- Simula una tarea pesada con un bucle de varios millones de iteraciones

```
// Simula tarea pesada: bucle de 5 segundos
const inicio = Date.now();
while (Date.now() - inicio < 5000) {}  // bloquea SOLO este hilo
```

- Genera un número ganador aleatorio entre 1 y 10
- Si el número es 7 → gana, si no → pierde
- Devuelve por **stdout** un JSON como `{ nombre: "Ana", numero: 7, premio: "¡Has ganado 1000€!" }`

02

Worker_threads

worker_threads: hilos dentro del mismo proceso

¿Qué es?

Crea un **hilo nuevo dentro del mismo proceso de Node**. Comparte memoria con el hilo principal pero corre en paralelo.

¿Cuándo usarlo?

- Cálculos pesados de CPU (procesar imágenes, encriptar datos, parsear ficheros grandes)
- Necesitas pasar objetos grandes sin serializar
- Quieres algo más ligero que un proceso completo

Nota: En el caso de worker, podemos seguir usando console.log() puesto que no cambiamos de proceso, seguimos en el mismo, no hay necesidad de implementar un protocolo diferente de comunicación.

server.js

```
import { Worker } from 'worker_threads';

app.get('/sumar', (req, res) => {
 const numeros = [1, 2, 3, 4, 5];

 // Crea el worker y le pasa los datos
 const worker = new Worker('./worker.js', {
   workerData: { numeros }            // ← pasa objetos directamente
(no solo strings)
 });

 // Escucha el resultado
 worker.on('message', (datos) => {
   res.json(datos);                   // → { resultado: 15 }
 });

 // Gestiona errores
 worker.on('error', (err) => {
   res.status(500).json({ error: err.message });
 });
});
```

worker.js

```
import { workerData, parentPort } from
'worker_threads';

// Recibe los datos del hilo principal
const { numeros } = workerData;

// Hace la tarea pesada
const resultado = numeros.reduce((acc, n) => acc + n,
0);

// Devuelve el resultado al hilo principal
parentPort.postMessage({ resultado });
```

```
# Terminal1 curl http://localhost:3001/sumar
# Terminal2 curl http://localhost:3001/ping
```

PROCESO PADRE (server.js)

1. ARRANQUE: ENVÍO DE DATOS INICIALES (process.argv)
[Argumentos serializados con JSON.stringify()]

PROCESO HIJO (tarea-pesada.js)

2. FLUJO DE DATOS: ENVÍO DE RESULTADO (stdout)
[Salida Estándar - Éxito como JSON]

3. FLUJO DE DATOS: NOTIFICACIÓN DE ERROR (stderr)
[Salida de Error Estándar - Fallo]

API QUE DELEGA LA TAREA pesado.js
Llama execFile, NO Bloquea el Event Loop
Atiende otras peticiones simultáneamente

```
import { execFile } from 'child_process';

app.get('/sumar', (req, res) => {
  const numeros = [1, 2, 3, 4, 5];
  // Lanza el script en un proceso separado
  execFile('node', ['tarea-pesada.js', JSON.stringify(numeros)],
    (error, stdout, stderr) => {
    if (error) {
      return res.status(500).json({ error: 'Falló el proceso' });
    }
    const datos = JSON.parse(stdout);
    res.json(datos);                // → { resultado: 15 }
  );
  // ✅ Mientras tanto, Node sigue atendiendo otras peticiones
});
```

SCRIPT DE TRABAJO PESADO
Ejecuta el Cálculo Aislado, Retorna JSON
Simula Tarea Pesada

```
// Este fichero se ejecuta en un proceso separado
const lista = process.argv[2]  // recibe datos por argument
  ? JSON.parse(process.argv[2])
  : [];

// Simulamos tarea pesada: suma todos los números
const resultado = lista.reduce((acc, n) => acc + n, 0);

// Devuelve el resultado al proceso padre
process.stdout.write(JSON.stringify({ resultado }));
```

NODE.JS

ESQUEMA DE COMUNICACIÓN PADRE-HIJO

Resumen

	child_process (Procesos)	worker_threads (Hilos)
¿Qué crea?	Un proceso de sistema operativo completamente nuevo e independiente (otra instancia de Node.js, Python, etc.).	Un nuevo hilo de ejecución ligero dentro del mismo proceso de Node.js.
Aislamiento de Memoria	Separada completamente. Los procesos no comparten memoria. La comunicación es costosa (IPC).	Compartida o Transferible. Pueden compartir memoria directamente (usando SharedArrayBuffer) para máxima velocidad.
Formato de Datos	Solo texto (strings). Obligatorio usar JSON.stringify() al enviar y JSON.parse() al recibir objetos.	Objetos directamente. Se pueden pasar objetos de JS, arrays, etc., sin serialización manual (copia o transferencia).
"Peso" (Recursos)	Pesado. Requiere arrancar una nueva instancia completa de Node.js (con su propio motor V8, libuv, etc.).	Ligero. Comparte el motor V8 y recursos del proceso padre, por lo que arranca y se gestiona más rápido.
Uso Típico	Ejecutar comandos de sistema (bash), scripts externos (Python), o aislar tareas críticas del todo.	Cálculo intensivo puro en JavaScript (criptografía, compresión, manipulación de imágenes/video).
Cuándo Usarlo	Para tareas independientes, aisladas o que no son de Node.js.	Para cálculos que necesitan muchos datos del servidor principal de forma rápida.
Imagen conceptual		

Ejercicio: El gimnasio de los hilos

Implementamos el backend de una app de fitness. Cuando un usuario pide su **plan de entrenamiento personalizado**, el servidor tiene que hacer un cálculo intensivo (calorías, series, repeticiones...) sin dejar de atender al resto de usuarios::

servidor.js con tres rutas:

- GET /entreno/:nombre → lanza un worker que calcula el plan (tarda ~3 segundos)
- GET /ping → responde inmediatamente con { servidor: "vivo" }
- GET /usuarios-activos → responde con cuántas peticiones /entreno hay en curso ahora mismo

calcular-entreno.js (el worker) que recibe { nombre, peso, altura } por workerData y:

- Simula cálculo pesado con un bucle de 6 segundos
- Calcula el IMC (peso / altura²)
- Según el IMC devuelve un plan diferente: "Volumen", "Definición" o "Mantenimiento"
- Devuelve `{ nombre, imc, plan, calorias }`

El reto: Lanzar tres peticiones a la vez:

- bashcurl http://localhost:3001/entreno/Ana?peso=65&altura=1.68 &
- curl http://localhost:3001/entreno/Pep?peso=90&altura=1.80 &
- curl http://localhost:3001/ping

Las tres deben responder, /ping la primera.

03

fork()

Multiproceso fork()

Hemos visto que cuando usamos child_process estamos levantando un nuevo Node, un nuevo proceso. Sin embargo no hay una comunicación entre procesos, el proceso “padre” espera a que el otro finalice para averiguar lo que ha ocurrido.

Usar **fork()** es como alquilar la cocina de al lado y contratar a un segundo Jefe de Cocina (otra instancia completa de Node.js).

A diferencia de otros métodos, la magia de **fork()** es que el dueño del restaurante (Proceso Padre) y el nuevo cocinero (Proceso Hijo) tienen un Walkie-Talkie integrado. **Pueden enviarse mensajes en tiempo real mientras trabajan.**

Caracteristicas

- Es exclusivo para Node.js: A diferencia de *spawn* o *exec* (que pueden ejecutar comandos de bash, Python, etc.), **fork()** está diseñado específicamente para ejecutar otros archivos JavaScript (.js).
- Tiene un canal de comunicación **bidireccional** (IPC): IPC significa **Inter-Process Communication**. Es el "walkie-talkie". Permite enviar mensajes de ida y vuelta usando eventos, no solo texto plano al final del proceso como con *child_process*.
- Es pesado: Arrancar un **fork()** levanta un nuevo motor V8 (interprete de JS) completo. No debes crear 1000 forks, sino uno por cada núcleo del procesador disponible, o solo para tareas de fondo específicas.

Multiproceso fork()

Asincronía Real: Si se hace una petición a /inventario, el navegador se quedará cargando unos segundos. Si en otra pestaña entramos a /ping, responderá al instante. **Esa es la magia de no bloquear el Event Loop!**

Vida útil del proceso: El fork() se hace dentro de *app.get()*. Esto significa que nace un **hijo por cada petición**, hace su trabajo, y luego "muere" con *process.exit()*. Si pusiéramos el fork arriba del todo, tendríamos un solo hijo vivo para siempre, lo cual es útil para otras arquitecturas, pero más complejo de gestionar si entran varias peticiones a la vez.

ESQUEMA DE COMUNICACIÓN NODE.JS:
API EXPRESS + FORK()

1. PETICIÓN HTTP:
GET /inventario

Cliente
(Navegador)

PROCESO PADRE
(server.js)

node.js
Express.js
Puerto 3000
Listening el Event Loop

No bloquea el Event Loop

```
app.get('/inventario', ...) {
  consto => {
    exito: '/inventario',
    exito: true,
    total_items: items,
  });
});
```

2. fork('tarea-larga.js')
[Nacimiento de Proceso]

3. .send({ comando: 'empezar_inventario' })
[Walkie-Talkie Padre]

CANAL IPC
(Walkie-Talkie /
Inter-Process Communication)

5. GET /ping
[Ruta Rápida]

6. res.json({ mensaje: '¡Servidor Atendiendo!' })
INMEDIATAMENTE

7. .send({ estado: 'Completado',
total: 5.000.000.000 })
[Walkie-Talkie Hijo]

Proceso Aislado de CPU

CPU

PROCESO HIJO
(tarea-larga.js)

CPU

```
{ comando:
  'empezar_inventario' }
```

```
process.on('message', ...) {
  for (i = 0; i < 5e9; ...) {
    ...
  }
};
```

```
process.send(...);
process.exit();
```

Worker

4. TRABAJO PESADO DE CPU
(Contando 5.000.000.000 iteraciones)

8. RESPUESTA HTTP:
res.json({ exito: true,
total_items: .... })

PROCESO PADRE
(server.js)

PROCESO HIJO
(tarea-larga.js)

Ejercicio: La Lotería

Usa el fork() para crear el proceso hijo que haga 3 tiradas.

servidor.js con Express que tenga dos rutas:

- GET /jugar/:nombre → lanza un proceso hijo para calcular si ese usuario gana en tres tiradas.

sorteo_tres.js (el proceso hijo) que recibe un nombre por argumento y:

- Simula una tarea pesada con un bucle de varios millones de iteraciones
- Genera 3 números aleatorios como máximo y va informando al servidor:
  - Primera tirada → "Primera tirada: El resultado ha sido 2 y ha perdido"
  - Segunda tirada → "Segunda tirada: El resultado ha sido 8 y ha perdido"
  - Tercera tirada → "Tercera tirada: El resultado ha sido 5 y ha perdido"
  - Resultado → "Pepe ha Perdido"

  - Primera tirada → "Primera tirada: El resultado ha sido 2 y ha perdido"
  - Segunda tirada → "Segunda tirada: El resultado ha sido 7 y ha ganado. No seguimos tirando"
  - Tercera tirada → "Tercera tirada: El resultado ha sido 5 y ha perdido"
  - Resultado → "Maria ha ganado"

04

exec() y spawn()

Multiproceso exec()

¿Qué es y cuándo se usa?

exec() abre una **terminal** de sistema (una Shell como bash en Linux o cmd en Windows) y ejecuta un comando. Acumula toda la respuesta en la memoria (en un **Buffer**) y te la devuelve al finalizar.

- **Ideal para:** Comandos rápidos que devuelven poco texto (saber la versión de un programa, listar un directorio pequeño, ver el espacio en disco).
- **Peligro**: Si el comando genera muchísima salida de texto, Node.js lanzará un error de maxBuffer y el proceso fallará.

En la cocina: Es el recadero urgente de la cocina

"Vete a buscar sal y aceite que nos falta URGENTEMENTE"

server.js

```
import {exec} from 'child_process';
app.get('/exec', (req, res) => {
   // Ejecutamos un comando del sistema (ej: listar archivos)
   // Windows: 'dir' | Linux/Mac: 'ls -la'
   exec('ls -la', (error, stdout, stderr) => {
       if (error) {
           console.error(`Error al ejecutar: ${error.message}`);
           res.json(`${error.message}`);
           return;
       }
       if (stderr) {
           console.error(`Avisos del sistema: ${stderr}`);
           res.json(`${stderr}`);
           return;
       }
       // stdout tiene TODO el texto junto, entregado de una vez
       console.log(`Resultado del comando:\n${stdout}`);
       res.json(`${stdout}`);
   });
});
```

Al ser lenguajes y procesos diferentes la comunicación se hará por el stdin, stderror y stdout, SOLO al principio y final de ejecutar el bash.

Multiproceso spawn()

¿Qué es y cuándo se usa?

spawn() ejecuta un comando, pero no espera a que termine para darte los datos. Te los va entregando en "pedacitos" (**chunks**) a través de un **Stream** (flujo de datos).

- **Ideal para:** Procesos pesados o continuos que generan muchísimos datos (comprimir un vídeo pesado, leer un log de servidor gigante, comandos que no terminan nunca como un ping continuo).
- **Ventaja**: Como no acumula todo en la memoria de golpe, nunca falla por exceso de tamaño (no hay límite de Buffer).

En la cocina: Es la necesidad de 10.000 litros de agua. Si los pedimos de golpe no sabremos como almacenarlos o transportarlos. Mejor una manguera y lo vamos usando poco a poco. STREAMING

server.js

```
import {spawn} from 'child_process';
app.get('/spawn', (req, res) => {
   // Le decimos al sistema: "Haz solo 5 pings y cierra la manguera"
   const procesoPing = spawn('ping', ['-c', '5', 'google.com']);
   // Preparamos la cabecera para que el navegador sepa que va a recibir datos poco a poco
   res.setHeader('Content-Type', 'text/plain; charset=utf-8');
   // Escuchamos el hilo de datos de salida
   procesoPing.stdout.on('data', (pedacito, contador) => {
       // Nos va llegando la información línea a línea o por bloques
       console.log(`Recibido: ${pedacito.toString()}`);
       res.write(`Dato recibido: ${pedacito.toString()}`); // Escribe en la respuesta sin
cerrarla
   });
   // Escuchamos si el proceso termina
   procesoPing.on('close', (codigo) => {
       console.log(`El proceso terminó con código ${codigo}`);
       res.end('Proceso ping terminado.'); // Cerramos la respuesta al cliente
   });
   procesoPing.stderr.on('data', (data) => {
       console.error(`Error en el hijo: ${data}`);
       res.status(500).json({ error: 'Fallo al hacer ping' });
   });
});
```

Necestitamos mantener nuestra respuesta abierta si queremos recibir el resultado de la ejecución y ver lo ocurrido.

Resumen

Característica	exec() (El Recadero)	spawn() (La Manguera)
Forma de entrega	De golpe al final (Buffer).	Poco a poco en directo (Stream).
Abre una Shell?	Sí (permite usar pipes |, redirecciones >).	No (más seguro y ligero, ejecutable directo).
Límite de datos	Límite máximo por defecto (1MB). Rompe si se pasa.	**Ilimitado. No consume memoria extra.**
Uso ideal	git status, ls, npm -v	Conversión de vídeo, logs gigantes, ping.
Tipo de evento	Usa un Callback tradicional (err, stdout) => {}.	Usa Eventos .on('data'), .on('close').
Imagen		

05

kill()

Bontón de pánico kill()

No importa si es el recadero (**exec**), la manguera (**spawn**) o el segundo cocinero (**fork**). Si la tarea se descontrola, el Jefe (Proceso Padre) tiene el derecho de despedir al trabajador inmediatamente y cerrar su cocina.

Cualquier objeto creado con **child_process** (ya sea vía spawn, exec, execFile o fork) es una instancia de la clase **ChildProcess** y todos heredan el método **.kill().**

¿Qué pasa cuando matamos a un hijo?

Se dispara el evento '**exit**' o '**close**'. Es el momento ideal para que el Padre limpie variables o avise al cliente: `res.send("Proceso cancelado por el administrador")`

Señales del Sistema

Por defecto, **.kill()** envía una señal llamada **SIGTERM** (terminación amable). Es como decir: "Oye, termina lo que estás haciendo y vete".

Si el proceso se queda "congelado" y no hace caso, podemos usar SIGKILL:

`proceso.kill('SIGKILL');` (Esto es el equivalente a "desenchufar la máquina" sin preguntar).

Ejercicio: Ping con contador

Modifica el código de Ping, haciendo un ping ilimitado y siendo tu el que cree un contador y cuando este llegue a 5,10, mates el proceso.

```
const procesoPing = spawn('ping', ['google.com']);
```